MyCodeSchool

Exerciții — funcții pe cifre

Python inițiere · proiectul `py-initiere` · Luca

Ce ai voie: `def`, `return` · `+ - * // %` · comparațiile · `and`, `or` · `if / elif / else` · `while`.
Încă NU folosim: `str(n)`, `len`, liste, `for`, `print` **în funcții**. Cifrele le scoți cu `%` și `//`, exact ca în C++.
Toate numerele primite sunt pozitive sau 0. Zero e un număr cu o cifră — funcțiile tale trebuie să-l știe.

Din C++ în Python: ce se schimbă

C++ (ce ai scris tu)	Python
`int ultimaCifra(int n) { ... }`	`def ultima_cifra(n: int) -> int:`
acolade `{ }`	**indentare** de 4 spații — ea arată ce e în funcție, în `while`, în `if`
`n / 10`	`n // 10`
`n = n / 10;`	`n //= 10`
`// comentariu`	`# comentariu`
nume `ultimaCifra`	nume `ultima_cifra` (cu `_`)

Atenție la `/`: în Python `4738 / 10` este `473.8`, nu `473`. Pentru „taie ultima cifră” scrii mereu `//`.

Atenție la `//` la început de linie: în Python nu e comentariu, e eroare. `functii.py` din proiectul tău începe acum cu `//` — șterge-l.

Cum lucrezi: `functii.py` calculează, `main.py` afișează

Fiecare exercițiu e o funcție în `functii.py` care **întoarce** rezultatul cu `return`. Nu afișează nimic. Afișarea se face doar în `main.py`, care arată așa (înlocuiește ce a pus PyCharm acolo):

```
from functii import ultima_cifra


def main():
    print(ultima_cifra(4738))
    print(ultima_cifra(0))


if __name__ == "__main__":
    main()
```

Ieșire așteptată:

```
8
0
```

La fiecare funcție nouă: o adaugi în `import` (`from functii import ultima_cifra, numar_cifre`) și pui câte un `print` pentru fiecare rând din tabelul ei.

Dacă vezi `None` în ieșire, funcția ta a **afișat** în loc să **întoarcă**: are `print` unde trebuia `return`.

La clasă

Ultima cifră 1

```
def ultima_cifra(n: int) -> int:
```

apel	întoarce
ultima_cifra(4738)	8
ultima_cifra(7)	7
ultima_cifra(0)	0

O singură linie, fără buclă: `return n % 10`.

Câte cifre are 2

```
def numar_cifre(n: int) -> int:
```

apel	întoarce
numar_cifre(4738)	4
numar_cifre(7)	1
numar_cifre(0)	1

Contor. Capcana e ultimul rând: cu `while n > 0` și contorul de la 0, pentru `0` întorci `0`. Pornește contorul de la `1` și taie cifre cât timp `n >= 10`.

Suma cifrelor 3

```
def suma_cifrelor(n: int) -> int:
```

apel	întoarce
suma_cifrelor(4738)	22
suma_cifrelor(1531)	10
suma_cifrelor(0)	0

*Acumulator pornit de la `0`, declarat înainte de buclă; `return` după buclă, **nu în ea**.*

Prima cifră 4

```
def prima_cifra(n: int) -> int:
```

apel	întoarce
prima_cifra(4738)	4
prima_cifra(7)	7
prima_cifra(0)	0

Tai cifre până rămâne una singură. Aceeași condiție ca la exercițiul 2.

Mai grele

Cifra maximă și cifra minimă 5

```
def cifra_maxima(n: int) -> int:
def cifra_minima(n: int) -> int:
```

apel	întoarce	apel	întoarce
cifra_maxima(4738)	8	cifra_minima(4738)	3
cifra_maxima(1531)	5	cifra_minima(1531)	1
cifra_maxima(0)	0	cifra_minima(305)	0

Pornește `maxim` (și `minim`) de la ultima cifră, `n % 10`, nu de la un număr ales de tine: la minim, dacă pornești de la `0`, întorci mereu `0`.

Câte cifre pare 6

```
def numar_cifre_pare(n: int) -> int:
```

apel	întoarce
numar_cifre_pare(4738)	2
numar_cifre_pare(1531)	0
numar_cifre_pare(0)	1

*Scoate cifra într-o variabilă, `cifra = n % 10`, și testează paritatea **cifrei**, nu a lui `n`. Zero e cifră pară: pentru `0` bucla `while n > 0` nu intră deloc, deci tratează-l înainte cu un `if`.*

Suma cifrelor pare 7

```
def suma_cifrelor_pare(n: int) -> int:
```

apel	întoarce
suma_cifrelor_pare(4738)	12
suma_cifrelor_pare(1531)	0
suma_cifrelor_pare(0)	0

Exercițiul 3 cu un `if` înăuntru.

Produsul cifrelor 8

```
def produsul_cifrelor(n: int) -> int:
```

apel	întoarce
produsul_cifrelor(4738)	672
produsul_cifrelor(305)	0
produsul_cifrelor(0)	0

Produsul pornește de la `1`, nu de la `0`. Tocmai de aceea, pentru `0` bucla nu intră și ai întoarce `1` — care nu e cifra lui. Tratează-l înainte.

De câte ori apare o cifră 9

```
def de_cate_ori_apare(n: int, cifra: int) -> int:
```

apel	întoarce
de_cate_ori_apare(1221, 2)	2
de_cate_ori_apare(4738, 9)	0
de_cate_ori_apare(100, 0)	2
de_cate_ori_apare(0, 0)	1

*Prima funcție cu **doi** parametri. Doar ultimul rând cere un `if` înainte de buclă: `n == 0 and cifra == 0`.*

Provocări

Fără indicii. Dacă te blochezi, trasează pe hârtie ce se întâmplă la fiecare rotire a buclei.

Numărul oglindit 10

```
def oglindit(n: int) -> int:
```

apel	întoarce
`oglindit(4738)`	`8374`
`oglindit(120)`	`21`
`oglindit(0)`	`0`

Palindrom 11

```
def este_palindrom(n: int) -> bool:
```

apel	întoarce
`este_palindrom(1221)`	`True`
`este_palindrom(4738)`	`False`
`este_palindrom(7)`	`True`

Corpul are **o singură linie**. Nu rescrie bucla de la exercițiul 10: cheam-o.

Rădăcina digitală 12

```
def radacina_digitala(n: int) -> int:
```

Faci suma cifrelor, apoi suma cifrelor rezultatului, până rămâne o singură cifră: `9875 → 29 → 11 → 2`.

apel	întoarce
`radacina_digitala(9875)`	`2`
`radacina_digitala(4738)`	`4`
`radacina_digitala(0)`	`0`

Ai scris deja suma cifrelor la exercițiul 3. Cheam-o, nu o scrie a doua oară.

Înainte să faci push, treci fiecare funcție prin astea:

1. Am pus în `main.py` câte un `print` pentru **fiecare rând** din tabel, inclusiv cel cu `0`? Ieșirea e identică cu coloana „întoarce”?
2. Apare `None` undeva? Atunci o funcție afișează în loc să întoarcă.
3. Am scris `/` undeva? În funcțiile astea trebuie peste tot `//`.
4. Am scris de două ori aceeași buclă? Dacă o funcție o are deja, o chem.